

Fundamentos de la programación 1

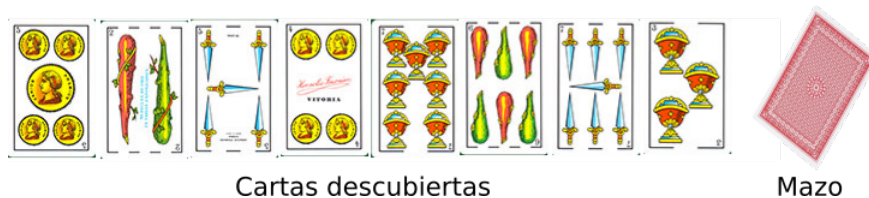
Grado en Desarrollo de Videojuegos

Examen de convocatoria ordinaria. Curso 23/24

Indicaciones generales:

- Se entregará únicamente el archivo `Program.cs` con el programa pedido. Se proporciona una **plantilla** con el esquema del programa y algunos métodos ya implementados.
- Las líneas 1 y 2 serán comentarios de la forma:
`// Nombre Apellidos`
`// Laboratorio, puesto`
- **Lee atentamente el enunciado** e implementa el programa tal como se pide, con los métodos, parámetros y requisitos que se especifican. No puede modificarse la representación propuesta, ni alterar los parámetros de los métodos especificados. No se especifica el modo de paso de los parámetros en los métodos (`out`, `ref`, ...), que debe determinar el alumno. Pueden implementarse todos los métodos adicionales que se consideren oportunos, especificando claramente su cometido, parámetros, etc.
- **El programa debe compilar y funcionar correctamente, y estar bien estructurado y comentado.** Se valorarán la claridad, la concisión y la eficiencia.
- La **entrega**: se realizará a través del servidor FTP de los laboratorios.
Importante: comprobar el archivo entregado en el puesto del profesor.

Vamos a implementar un solitario de cartas con baraja española: 4 palos (oros, copas, espadas y bastos), cada uno con 10 cartas (40 cartas en total). El juego comienza barajando el mazo las cartas y poniendo sobre la mesa un número fijado de ellas descubiertas (por ejemplo 8). Si entre ellas hay dos con mismo valor pueden retirarse y colocarse otras dos del mazo en su lugar. Se seguirán retirando parejas hasta que se hayan emparejado todas las cartas (el jugador gana), o bien se alcance un estado en el que queden cartas pero no haya ninguna pareja (el jugador pierde). Un estado del juego puede ser el siguiente (en este caso el jugador puede elegir retirar la pareja de cincos o la pareja de setes):



En la implementación representaremos cada carta con un único entero de 0 a 39, agrupado los palos del siguiente modo:

$$\begin{array}{cccc} \underbrace{0 \dots 9} & \underbrace{10 \dots 19} & \underbrace{20 \dots 29} & \underbrace{30 \dots 39} \\ \text{oros} & \text{copas} & \text{espadas} & \text{bastos} \end{array}$$

Así, dada una carta k , su valor viene dado por $k \% 10$ y su palo por $k / 10$ (de este modo: 0=oros, 1=copas, 2=espadas, 3=bastos). Por ejemplo, 36 será el 6 de bastos; 3 representa el 3 de oros y 0 es el 0 de oros. Nótese que según esta representación, los valores de las cartas van de 0 a 9 (y no de 1 a 10, como en la baraja tradicional).

El estado del juego y la inicialización del mazo se dan ya implementados en la plantilla:

```
class SolitarioParejas{
    static Random rnd = new Random(); // generador de aleatorios (se usará al final)
    const int DESCUBIERTAS = 8;        // número de cartas descubiertas en la mesa
    static void Main() {
        int [] mazo = new int[40],      // array de cartas del mazo
            mesa = new int[DESCUBIERTAS]; // array de cartas de la mesa
        int prim; // posición de la primera carta aún no extraída del mazo
        InicializaMazo(mazo, out prim); // método ya implementado que genera el mazo
    }
}
```

Las cartas descubiertas en la mesa se representan con el array `mesa`, cuyo tamaño viene dado por la constante `DESCUBIERTAS`. Para indicar que una posición no tiene carta se utilizará el valor -1 en el array.

El mazo es otro array, `mazo`, junto con un entero `prim`, que indica la posición de la primera carta disponible. Inicialmente será `prim=0`, y a medida que se vayan sacando cartas a la mesa se irá incrementando este índice. De este modo, los valores a la izquierda de `prim` no son relevantes (representan cartas que se han sacado a la mesa y ya no están en `mazo`).

El método `InicializaMazo(int [] mazo, out int prim)` se da ya implementado y rellena el mazo con las 40 cartas en un orden fijado (ver este método en la plantilla). Para acabar de entender la representación propuesta, si tomamos el mazo que genera este método y sacamos 8 cartas a la mesa, el estado del juego será:

```
mazo = [11,36,1,34,12,7,38,24,23,2,14,33,8,22,16,...
mesa = [11,36,1,34,12,7,38,24]
prim = 8
```

Como `prim = 8`, la primera carta del mazo será 23. En este estado, podemos retirar la pareja 11, 1 (1 de copas, 1 de oros), con lo que tendremos (el mazo queda igual):

```
mesa = [23,36,2,34,12,7,38,24]
prim = 10
```

El 11 se reemplaza por 23 (la siguiente del mazo) y `prim` avanza a 9; el 1 se reemplaza por 2 y `prim` avanza a 10.

Para desarrollar el juego implementaremos los siguientes métodos:

- (1) `void ExtraeCarta(int [] mesa, int pos, int [] mazo, int prim)`: extrae del mazo la primera carta disponible (la de la posición `prim`) y la coloca en la mesa, en la posición `pos` (sobrescribe el valor de `mesa` en `pos`); además, incrementa el valor `prim`. Si el mazo está vacío deja vacía la posición `pos` de la mesa y no altera `prim`.
- (1) `void InicializaMesa(int [] mazo, int prim, int [] mesa)`: utilizando el método anterior, rellena la mesa con cartas que irá extrayendo del mazo (el número de cartas a extraer es `DESCUBIERTAS`).
- (1) `void Render(int [] mesa, int prim)`: muestra en pantalla el estado del juego con el siguiente aspecto (capturas de dos estados de juego):

```
Posiciones: 0 1 2 3 4 5 6 7
Cartas:      1 6 1 4 2 7 8 4
Quedan 32 cartas en el mazo
```

```
Posiciones: 0 1 2 3 4 5 6 7
Cartas:      5 6 5 . 4 4 . 6
Quedan 0 cartas en el mazo
```

La primera línea indica las posiciones de las cartas sobre la mesa para facilitar luego la selección de pares. La segunda línea muestra las cartas de la mesa, con su valor numérico y un color que indica el palo: oros amarillo, copas rojo, espadas azul y bastos verde (el valor y el palo se obtienen como se ha explicado arriba). Para establecer el color amarillo haremos: `Console.ForegroundColor = ConsoleColor.Yellow` (análogo para otros colores).

Para las posiciones vacías de la mesa (con valor -1) se escribirá el carácter '·'. Además, se indica el número de cartas que quedan aún en el mazo.

- (1) `void PidePosiciones(int p1, int p2)`: solicita de teclado dos posiciones y las devuelve en `p1` y `p2`. Debe asegurarse de que cada uno de esos valores está en el rango `[0,DESCUBIERTAS)`; en otro caso volverá a pedirlo, hasta obtener un valor válido.
- (1) `bool HayPar(int [] mesa)`: determina si `mesa` contiene alguna pareja de cartas del mismo valor.

Con estos métodos ya puede implementarse una primera versión del juego. El bucle principal, en cada iteración, solicitará dos posiciones de la mesa (con `PidePosiciones`); si contienen cartas del mismo valor, las retirará de la mesa (con `ExtraeCarta`) y mostrará el estado (con `Render`). Terminará cuando no haya parejas en la mesa (se comprueba con `HayPar`).

A continuación, implementaremos los siguientes métodos:

- (1) `void Salva(int [] mesa, int [] mazo, int prim)`: guarda el estado del juego en el archivo "solitario", en tres líneas de texto. En la primera guarda los valores de `mesa`, en la segunda los valores de `mazo` (en ambos casos separados por un blanco) y en la tercera guarda el valor `prim`. Para el estado inicial del juego guardaría:

```
11 36 1 34 12 7 38 24
11 36 1 34 12 7 38 24 23 2 14 33 8 22 16 ...
8
```

- (1) `void Lee(int [] mesa, int [] mazo, int prim)`: lee del archivo "solitario" el estado del juego, guardado en el formato del método anterior, y lo devuelve en los tres parámetros indicados. Para leer una línea de números será útil la siguiente instrucción:

```
string [] nums = (f.ReadLine()).Split(" ",StringSplitOptions.RemoveEmptyEntries);
```

Por ejemplo, para la primera línea del archivo de arriba devolverá el array de `strings`:

```
nums = ["11","36","1","34","12","7","38","24"]
```

- (1) `void InicializaMazoAleatorio(int [] mazo, int prim)`: genera en `mazo` una permutación aleatoria de las cartas (reemplazará a `InicializaMazo` enriqueciendo el juego).

Para ello, comienza rellenando `mazo` con las cartas (valores de 0 a 39) y después las desordena con el siguiente algoritmo: recorre el vector de principio a fin intercambiando cada carta con una de las siguientes (incluida ella misma), elegida de manera aleatoria. El generador `rnd` está definido al principio de la plantilla y la expresión `rnd.Next(a,b)` devuelve un valor aleatorio del intervalo `[a,b)`.

- (1) `bool MesaVacía(int [] mesa)`: comprueba si la mesa está vacía (solo ocurre cuando se han emparejado todas las cartas).
- (1) `void Main()`: completa el juego utilizando los métodos anteriores.
 - Comenzará preguntado al usuario si quiere generar nuevo juego (genera el estado con `InicializaMazoAleatorio` y `InicializaMesa`) o continuar una partida previamente guardada (lo leerá de archivo con `Lee`).
 - El bucle principal funciona esencialmente igual que antes (ver arriba), pero además permitirá abortar el juego cuando el jugador introduzca la misma posición en `PidePosiciones`, es decir, que devuelva el mismo valor en `p1` y `p2`.
 - Tras terminar el bucle, si el jugador ha abortado el juego preguntará si quiere guardar partida (con `Salva`); en caso contrario comprobará e informará si el jugador ha ganado o perdido (se comprueba con `MesaVacía`).